

Guía Manual de Scripting Personalizado

Guía Completa para C# (Roslyn) y JavaScript (Jint)

FileFlow Studio — Motor Visual de Automatización DAG en .NET 10

⚡ FileFlow Studio — Manual de Scripting Personalizado (C# & JavaScript)

Bienvenido a la guía oficial del nodo **Script Personalizado** (`CustomScriptNode`). Este documento está diseñado paso a paso para que cualquier usuario, desde un nivel principiante hasta avanzado, pueda escribir scripts a medida, crear salidas inteligentes, consultar metadatos del flujo y transformar archivos sin necesidad de instalar entornos de desarrollo adicionales.

Tabla de Contenidos

1. ¿Qué es el Nodo de Script Personalizado?
2. Elegir entre C# y JavaScript
3. La Ventana del Estudio de Scripting
4. Creación y Gestión de Puertos Dinámicos (Inputs y Outputs)
5. Acceso a la Información del Archivo (`Item` / `file`)
6. Acceso a Variables Implícitas y Metadatos del Flujo
7. Cómo Emitir Archivos hacia las Salidas (`EmitAsync` y `emit`)
8. Mensajes de Registro y Telemetría (`Log` y `console.log`)
9. Catálogo de 7 Ejemplos Prácticos Listos para Usar
10. Preguntas Frecuentes y Solución de Errores

1. ¿Qué es el Nodo de Script Personalizado?

El nodo **Script Personalizado** es una estación de trabajo programable que se coloca en cualquier punto del pipeline visual de FileFlow Studio. Cuando un archivo llega al nodo, el motor ejecuta el código que hayas escrito y te permite:

- **Inspeccionar y clasificar** el archivo según su tamaño, nombre, extensión o contenido.
- **Acceder a metadatos** generados por nodos anteriores (hashes calculados, datos de cámara EXIF, número de páginas de un PDF, duración de vídeo, etc.).
- **Modificar propiedades o añadir etiquetas** (*Tags*) que otros nodos podrán leer más adelante.
- **Bifurcar el flujo** enviando el archivo hacia una o múltiples salidas personalizadas (por ejemplo: `Aprobados` , `Grandes` , `Errores` , `Documentos`).

2. Elegir entre C# y JavaScript

FileFlow Studio incorpora **dos motores de ejecución independientes**:

Característica	⚡ C# (Roslyn Scripting)	🌐 JavaScript (Jint Sandbox)
---	---	---
¿Para quién es?	Quienes buscan máximo rendimiento o familiaridad con C#/ .NET.	Quienes prefieren sintaxis sencilla y flexible tipo web.
Rendimiento	Ultra-rápido (Compilado JIT en memoria de .NET 10 con cacheo).	Rápido y seguro (Sandbox administrado en memoria).
Manejo Asíncrono	<code>await EmitAsync("Salida");</code>	<code>emit("Salida", item);</code>
Ideal para...	Cálculos matemáticos, manipulación de archivos y tipos .NET.	Manipulación de strings, expresiones regulares y JSON.

Nota: Puedes cambiar de lenguaje en cualquier momento desde el selector en la parte superior de la ventana del editor.

3. La Ventana del Estudio de Scripting

Para abrir el estudio de programación, haz clic en el botón  **Editor de Scripts ...** presente en la tarjeta del nodo en el lienzo visual.

La ventana cuenta con 4 áreas principales:

- Barra Superior:** Selector de lenguaje (C# / JavaScript), botón  **Manual PDF ...** y menú de **Plantillas predefinidas**.
- Editor Central:** Editor con numeración de líneas, coloreado de sintaxis profesional y detección visual de texto.
- Panel Lateral Derecho:**
 - Pestaña 🚫 Puertos:** Añade o elimina puertos de entrada y salida con un solo clic.
 - Pestaña 🧪 Probar en Vivo:** Simula la ejecución de tu script con un archivo de prueba y observa en tiempo real los mensajes de log y los puertos por los que sale el archivo.
 - Pestaña 📁 Guardar:** Guarda tus scripts favoritos en tu biblioteca personal para reutilizarlos en otros flujos.
- Pie de Ventana:** Botón verde  **✓ Aplicar y Cerrar** para sincronizar los cambios en el pipeline.

4. Creación y Gestión de Puertos Dinámicos (Inputs y Outputs)

A diferencia de otros nodos con entradas y salidas fijas, en el nodo de script **tú decides qué puertos existen**.

¿Cómo añadir un nuevo puerto?

- En la pestaña  **Puertos**, escribe el nombre deseado en la caja de texto (por ejemplo `Comprimidos`).
- Pulsa el botón  **+ Añadir**.
- El puerto aparecerá inmediatamente en la lista y, al aplicar, se creará el conector en la tarjeta del nodo en el lienzo visual para que puedas arrastrar un cable hacia el siguiente nodo.

Reglas para los nombres de puertos:

- Usa nombres claros sin espacios raros (ejemplos: `Out`, `True`, `False`, `Grandes`, `Videos`, `PDFs`, `Cuarentena`).
- El puerto de salida por defecto suele llamarse `Out`.

5. Acceso a la Información del Archivo (`Item` / `file`)

Dentro de tu script, el archivo entrante está disponible en la variable global `Item` (o `file`).

Propiedades Principales Disponibles

Propiedad	Tipo	Descripción	Ejemplo de Uso en C#	Ejemplo de Uso en JS
---	---	---	---	---
<code>Item.FileName</code>	<code>string</code>	Nombre del archivo con su extensión.	<code>string n = Item.FileName;</code>	<code>let n = item.FileName;</code>
<code>Item.CurrentPath</code>	<code>string</code>	Ruta completa actual del archivo en disco.	<code>string p = Item.CurrentPath;</code>	<code>let p = item.CurrentPath;</code>
<code>Item.OriginalPath</code>	<code>string</code>	Ruta original con la que entró al pipeline.	<code>string o = Item.OriginalPath;</code>	<code>let o = item.OriginalPath;</code>
<code>Item.FileSizeBytes</code>	<code>long</code>	Tamaño exacto del archivo en bytes.	<code>long bytes = Item.FileSizeBytes;</code>	<code>let bytes = item.FileSizeBytes;</code>
<code>Item.IsDirectory</code>	<code>bool</code>	Indica si el elemento es una carpeta.	<code>if (Item.IsDirectory) ...</code>	<code>if (item.IsDirectory) ...</code>
<code>Item.Metadata</code>	<code>Dictionary</code>	Diccionario clave-valor con metadatos.	<code>Item.Metadata["Clave"] = 123;</code>	<code>item.Metadata['Clave'] = 123;</code>
<code>Item.Tags</code>	<code>HashSet</code>	Colección de etiquetas de texto.	<code>Item.Tags.Add("Revisado");</code>	<code>item.Tags.Add("Revisado");</code>

6. Acceso a Variables Implícitas y Metadatos del Flujo

FileFlow Studio propaga metadatos de forma automática entre nodos. Puedes acceder a ellos de tres maneras:

1. Acceso Directo al Diccionario `Item.Metadata`

Si en nodos anteriores calculaste el Hash SHA-256 o extrajiste metadatos EXIF, puedes leerlos directamente:

```
// En C#:
if (Item.Metadata.TryGetValue("Hash:SHA256", out var hash))
{
    Log($"El hash SHA256 es: {hash}");
}
```

```
// En JavaScript:
if (item.Metadata['Hash:SHA256']) {
    log("El hash SHA256 es: " + item.Metadata['Hash:SHA256']);
}
```

2. Función Universal `Resolve("{Plantilla}")`

Puedes resolver cualquier token o fórmula de FileFlow mediante la función `Resolve( ... )` (en C#) o `resolve( ... )` (en JS):

```
// En C#:
string nombreFormateado = Resolve("{Year}-{Month}_{FileNameNoExt}_{OK}.{Ext}");
string tamanoTexto = Resolve("El archivo pesa {SizeMB} MB");
```

```
// En JavaScript:
let nombreFormateado = resolve("{Year}-{Month}_{FileNameNoExt}_{OK}.{Ext}");
let usuarioActual = resolve("{UserName}");
```

3. Catálogo de Variables Implícitas Disponibles

Token Implícito	Descripción	Ejemplo de Resultado
<code>---</code>	<code>---</code>	<code>---</code>
<code>{FileName}</code>	Nombre de archivo con extensión.	<code>foto_vacaciones.jpg</code>
<code>{FileNameNoExt}</code>	Nombre de archivo sin extensión.	<code>foto_vacaciones</code>
<code>{Ext}</code> o <code>{Extension}</code>	Extensión sin punto.	<code>jpg</code>
<code>{SizeMB}</code>	Tamaño formateado en Megabytes.	<code>14.50</code>
<code>{SizeBytes}</code>	Tamaño en bytes enteros.	<code>15204352</code>
<code>{SizeGB}</code>	Tamaño en Gigabytes.	<code>1.42</code>
<code>{Year}</code> , <code>{Month}</code> , <code>{Day}</code>	Año, mes y día actual.	<code>2026</code> , <code>09</code> , <code>01</code>
<code>{Date:yyyy-MM-dd}</code>	Fecha actual con formato a medida.	<code>2026-09-01</code>
<code>{UserName}</code>	Usuario actual del sistema operativo.	<code>kaoticos53</code>
<code>{MachineName}</code>	Nombre del equipo / ordenador.	<code>DESKTOP-PRO</code>
<code>{Hash:SHA256}</code>	Hash SHA-256 (si se calculó previamente).	<code>e3b0c44298fc1c ...</code>
<code>{Exif:CameraModel}</code>	Modelo de cámara EXIF (si es imagen).	<code>Sony ILCE-7M4</code>
<code>{Doc:PageCount}</code>	Número de páginas (si es PDF/Doc).	<code>18</code>
<code>{Media:Duration}</code>	Duración de audio o vídeo.	<code>00:45:12</code>

7. Cómo Emitir Archivos hacia las Salidas (`EmitAsync` y `emit`)

Para que el archivo continúe su viaje hacia los siguientes nodos conectados en el diagrama, **debes emitirlo hacia uno o varios puertos de salida.**

Sintaxis en C#:

```
// Emitir al puerto predeterminado "Out"
await EmitAsync("Out");

// Emitir a un puerto personalizado
await EmitAsync("Aprobados");

// Emitir a múltiples puertos (bifurcación en paralelo)
await EmitAsync("CopiaLocal");
await EmitAsync("CopiaNube");
```

Sintaxis en JavaScript:

```
// Emitir al puerto predeterminado "Out"
emit("Out", item);

// Emitir a un puerto personalizado
emit("Aprobados", item);

// Bifurcación condicional
if (item.FileSizeBytes > 10485760) {
    emit("Grandes", item);
} else {
    emit("Pequenos", item);
}
```

8. Mensajes de Registro y Telemetría (`Log` y `console.log`)

Todos los mensajes que envíes desde tu script aparecerán en el panel de telemetría inferior de FileFlow Studio y en el probador en vivo:

- **En C#:**
- `Log("Operación completada");` (Nivel Información)
- `Log("Advertencia: tamaño sospechoso", LogLevel.Warning);`
- `Log("Error al validar contenido", LogLevel.Error);`
- **En JavaScript:**
- `console.log("Mensaje informativo");`
- `console.warn("Mensaje de advertencia");`
- `console.error("Mensaje de error");`
- `log("Mensaje rápido");`

9. Catálogo de 7 Ejemplos Prácticos Listos para Usar

🚀 Ejemplo 1 (C# - Nivel Básico): Clasificador por Tamaño en Megabytes

Puertos de Salida requeridos: `Grandes` , `Pequenos`

```
// Calcula el tamaño en Megabytes
double tamanoMb = Item.FileSizeBytes / (1024.0 * 1024.0);

// Guarda el tamaño en los metadatos para otros nodos
Item.Metadata["TamanoMB_Calculado"] = tamanoMb;

if (tamanoMb >= 100.0)
{
    Item.Tags.Add("Pesado");
    Log($"Archivo grande detectado ({tamanoMb:F2} MB) -> Enviando a Grandes");
    await EmitAsync("Grandes");
}
else
{
    Item.Tags.Add("Ligero");
    Log($"Archivo ligero ({tamanoMb:F2} MB) -> Enviando a Pequenos");
    await EmitAsync("Pequenos");
}
}
```

🚀 Ejemplo 2 (C# - Nivel Básico): Inyección de Auditoría y Metadatos de Sistema

Puertos de Salida requeridos: Out

```
// Registra quién y cuándo procesó el archivo
Item.Metadata["Auditoria_Usuario"] = Environment.UserName;
Item.Metadata["Auditoria_Equipo"] = Environment.MachineName;
Item.Metadata["Auditoria_FechaUtc"] = DateTime.UtcNow.ToString("yyyy-MM-dd HH:mm:ss");

Item.Tags.Add("Auditado");
Log($"Auditoría inyectada en {Item.FileName}");

await EmitAsync("Out");
```

🚀 Ejemplo 3 (C# - Nivel Medio): Enrutador Triple por Extensión

Puertos de Salida requeridos: Imágenes, Videos, Documentos, Otros

```

var ext = Path.GetExtension(Item.FileName).ToLowerInvariant();

var imagenes = new[] { ".jpg", ".jpeg", ".png", ".webp", ".gif", ".raw" };
var videos = new[] { ".mp4", ".mkv", ".mov", ".avi", ".webm" };
var docs = new[] { ".pdf", ".docx", ".xlsx", ".txt", ".epub" };

if (imagenes.Contains(ext))
{
    Item.Metadata["TipoContenido"] = "Imagen";
    await EmitAsync("Imagenes");
}
else if (videos.Contains(ext))
{
    Item.Metadata["TipoContenido"] = "Video";
    await EmitAsync("Videos");
}
else if (docs.Contains(ext))
{
    Item.Metadata["TipoContenido"] = "Documento";
    await EmitAsync("Documentos");
}
else
{
    Item.Metadata["TipoContenido"] = "Otros";
    await EmitAsync("Otros");
}

```

✦ Ejemplo 4 (JavaScript - Nivel Básico): Sanitizador y Validador de Nombres

Puertos de Salida requeridos: `Validos`, `RequierenLimpieza`

```

var nombre = item.FileName;

// Comprueba si contiene caracteres extraños o dobles espacios
var tieneEspaciosDobles = nombre.indexOf(' ') !== -1;
var tieneCaracteresProhibidos = /[!@?;|!#$%&*]/.test(nombre);

if (tieneEspaciosDobles || tieneCaracteresProhibidos) {
    console.warn("El archivo contiene caracteres inválidos: " + nombre);
    item.Metadata["NombreSugerido"] = nombre.replace(/\\s+/g, '_').replace(/[!@?;|!#$%&*]/g, '');
    item.Tags.Add("CorregirNombre");
    emit("RequierenLimpieza", item);
} else {
    emit("Validos", item);
}

```

✦ Ejemplo 5 (JavaScript - Nivel Medio): Uso de Plantillas y Enrutador Multimedia

Puertos de Salida requeridos: `Multimedia`, `Resto`

```

var ext = item.FileName.split('.').pop().toLowerCase();
var esMedia = ['mp4', 'mkv', 'mp3', 'wav', 'flac'].indexOf(ext) !== -1;

if (esMedia) {
    // Resolver plantilla de fecha y nombre
    var carpetaDestino = resolve("{Year}/{Month}/Media");
    item.Metadata["CarpetaOrganizada"] = carpetaDestino;

    console.log("Archivo multimedia organizado en: " + carpetaDestino);
    emit("Multimedia", item);
} else {
    emit("Resto", item);
}

```

✦ Ejemplo 6 (C# - Nivel Avanzado): Verificación de Hash Previo y Control de Calidad

Puertos de Salida requeridos: `Verificados`, `SinHash`

```

// Comprueba si un nodo anterior (HashCalculatorNode) generó la firma SHA256
if (Item.Metadata.TryGetValue("Hash:SHA256", out var hashObj) && hashObj != null)
{
    string hash = hashObj.ToString();
    Log($"Hash verificado: {hash[..8]}... para {Item.FileName}");

    Item.Metadata["EstadoIntegridad"] = "OK";
    Item.Tags.Add("HashVerificado");

    await EmitAsync("Verificados");
}
else
{
    Log($"Advertencia: {Item.FileName} no tiene hash previo", LogLevel.Warning);
    Item.Metadata["EstadoIntegridad"] = "Pendiente";
    await EmitAsync("SinHash");
}

```

✦ Ejemplo 7 (JavaScript - Nivel Avanzado): Clasificación de Documentos por Páginas

Puertos de Salida requeridos: `InformesCortos`, `LibrosExtensos`, `NoDocumento`

```

// Comprueba si DocumentProcessorNode extrajo el número de páginas
if (item.Metadata['Doc:PageCount']) {
    var paginas = parseInt(item.Metadata['Doc:PageCount'], 10);

    if (paginas <= 10) {
        item.Tags.Add("InformeCorto");
        emit("InformesCortos", item);
    } else {
        item.Tags.Add("LibroExtenso");
        emit("LibrosExtensos", item);
    }
} else {
    emit("NoDocumento", item);
}

```

10. Preguntas Frecuentes y Solución de Errores

? ¿Qué ocurre si mi script no llama a `EmitAsync` ni a `emit` ?

Si el script no emite el archivo por ningún puerto, el archivo se considerará filtrado/descartado y no continuará hacia los nodos siguientes.

? ¿Qué significa el error `CS1002: ; expected` ?

En C#, cada instrucción debe terminar obligatoriamente con un punto y coma `;`. Revisa que todas tus líneas terminen con `;`.

? ¿Qué pasa si mi script tarda demasiado tiempo?

Por seguridad, el nodo cuenta con un parámetro de **Timeout (por defecto 30 segundos)**. Si tu script entra en un bucle infinito o se bloquea, el motor cancelará la ejecución y registrará un error en el archivo sin colgar la aplicación.

? ¿Puedo emitir un archivo hacia múltiples salidas a la vez?

Sí. Puedes llamar varias veces a `await EmitAsync("Salida1");` y `await EmitAsync("Salida2");` en C# o `emit("Salida1", item);` y `emit("Salida2", item);` en JavaScript para duplicar el flujo en ramas paralelas.

FileFlow Studio — Documentación Oficial de Scripting.